

Git & GitHub, einmal in Ruhe erklärt

Damit du an der Website weiterarbeiten kannst, ohne dass etwas verloren geht. Kein Entwickler-Wissen nötig, nur die Begriffe, die dir in Cloud Code und auf GitHub begegnen.

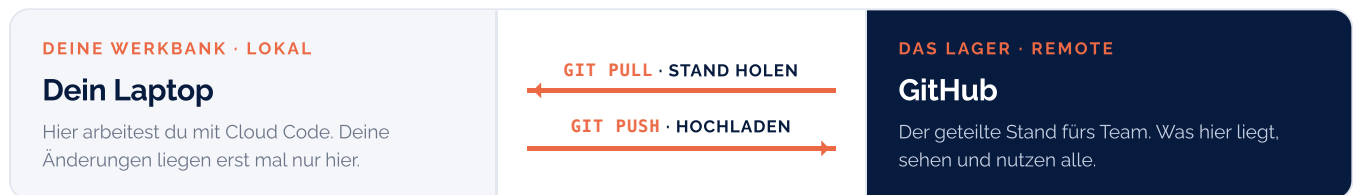
DIE EINE REGEL, DIE ZÄHLT

Jede Session beginnt mit **git pull** und endet mit **git push**.

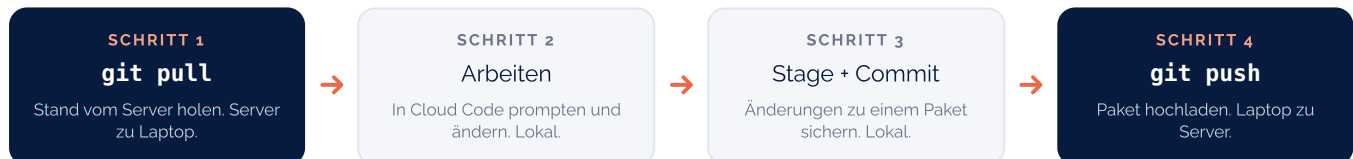
Pull holt dir zu Beginn den aktuellen Stand vom Server. **Push** sichert deine Arbeit am Ende dort wieder. Wenn du den Push vergisst, liegt deine Arbeit nur auf deinem Laptop und niemand sonst sieht sie. Alles andere auf diesem Blatt ist Beiwerk. Das hier ist das Wichtige.

// DAS GROSSE BILD

Stell dir **GitHub** als gemeinsames Lager vor und deinen **Laptop** als deine Werkbank. Du holst dir den Stand, arbeitest lokal, und schickst das Ergebnis zurück. Erst nach dem Zurückschicken sehen die anderen im Team deine Änderungen. Solange etwas nur auf deiner Werkbank liegt, sieht es niemand.



// DER TÄGLICHE ABLAUF



Das Gute: Cloud Code macht die Schritte 3 und 4 für dich, wenn du am Ende einfach schreibst „committe und pushe“. Du musst dir die Befehle nicht merken. Du musst nur **verstehen, was dabei passiert**, und am Ende dran denken, dass gepusht wird. Sonst ist die Arbeit nicht gesichert.

// UND WENN DU DICH VERRENNST?

Kein Problem, hier kann fast nichts kaputtgehen. Jeder Commit ist gespeichert, also lässt sich jederzeit ein früherer Stand wiederherstellen (Rollback). Wir arbeiten hier bewusst frei, wie in einem Sandkasten. Trau dich also ruhig, Sachen auszuprobieren. Und wenn eine Frage offen ist: Frag einfach deine Claude-Session, die erklärt dir jeden dieser Begriffe genauso wie hier, nur auf deinen konkreten Fall bezogen.

// DIE BEGRIFFE, KURZ UND SCHMERZLOS

Repository

Repo

DAS GANZE

Das komplette Projekt samt seiner gesamten Änderungshistorie. Unser Repo für die Website heißt **cognicore.de**.

Pull

git pull

HOLEN

Lädt die neuesten Änderungen von GitHub auf deinen Rechner. Immer zu Beginn, sonst arbeitest du auf einem veralteten Stand.

Commit

git commit

SICHERN

Fasst die gestageten Änderungen zu einem gespeicherten Paket zusammen, mit kurzer Beschreibung (Commit-Message). Bleibt zunächst **lokal** bei dir.

main

HAUPTSTAND

Der Branch, auf dem die echte Website liegt. Das, was gilt. Bei uns arbeiten alle meistens direkt hier.

Merge

git merge

ZUSAMMENFÜHREN

Bringt die Änderungen eines Branches in einen anderen, zum Beispiel deinen Branch zurück in **main**.

Rollback

ZURÜCKSPULEN

Weil jeder Commit gespeichert ist, kann man jederzeit zu einem früheren Stand zurück. Nichts geht wirklich verloren.

Merge-Konflikt

KOLLISION

Wenn zwei Leute dieselbe Zeile unterschiedlich geändert haben, fragt Git, welche Version gelten soll. Kein Drama, einmal kurz entscheiden.

Lokal vs. Remote

WO LIEGT ES

Lokal ist die Kopie auf deinem Rechner. **Remote** ist die gemeinsame Version auf GitHub, mit der alle arbeiten.

Staging

git add

VORBEREITEN

Du wählst aus, welche geänderten Dateien in den nächsten Commit sollen. Sie sind dann „gestaged“, also fürs Sichern aufs Tablett gelegt.

Push

git push

HOCHLADEN

Schickt deine lokalen Commits zu GitHub. **Erst jetzt** ist deine Arbeit gesichert und für alle anderen sichtbar.

Branch

PARALLELKOPIE

Eine abgezwigte Arbeitskopie, um etwas Größeres auszuprobieren, ohne **main** anzufassen. Später wird sie wieder zusammengeführt.

Pull Request

PR

FREIGABE

In großen Teams üblich: Du schlägst vor, einen Branch in **main** zu übernehmen, jemand schaut drüber und gibt frei. Für unsere Website brauchen wir das normalerweise nicht.

Diff

WAS SICH ÄNDERTE

Die Vorher-Nachher-Ansicht einer Datei. **Rot** sind entfernte Zeilen, **Grün** sind neue Zeilen.

.gitignore

BLEIBT DRAUSSEN

Eine Liste von Dateien, die bewusst nicht hochgeladen werden. Deshalb bleibt zum Beispiel deine lokale **.claude** auf deinem Rechner und landet nie auf dem Server.

DREI SÄTZE, DIE DIR VERWIRRUNG ERSPAREN

Push ist nicht Merge

Push bewegt Code zwischen deinem Rechner und GitHub. **Merge** bewegt Code zwischen zwei Branches. Zwei völlig verschiedene Richtungen.

Committed heißt noch nicht hochgeladen

Ein Commit liegt erst mal nur lokal bei dir. Erst der **push** bringt ihn zu GitHub. Beides sind zwei getrennte Schritte.

Nichts ist live ohne Push

Eine Änderung nur auf deinem Laptop sieht niemand. „Ist online“ stimmt erst, wenn du gepusht hast. Vorher wäre es geschwindelt.